KARNAVATI UNIVERSITY
UNIVERSITY
NAAC GRADE A+

KARNAVATI UNIVERSITY
UNITEDWORLD INSTITUTE OF TECHNOLOGY (UIT)

MINI PROJECT REPORT

ON

"End-to-End Mini Compiler with Compiler Phase Visualization"

Submitted as a part of the course

Compiler Design

Submitted By

Student Name	Enrollment No.
Hitik Shah	202307020106
Het Sharma	202307020045
Manya Singh	202307020133
Yashvi Muchhala	202307020150

Program: B.Tech CSE
Semester: VII
Section: CSE

Under the Guidance of

Dr. Bela Shrimali
Associate Professor
Department of Computer Science & Engineering Technology

Karnavati University, Gandhinagar

TABLE OF CONTENTS

ABSTRACT 4
CHAPTER 1 – INTRODUCTION 5
1.1 Introduction 5
1.2 Background 5
1.3 Problem Statement 6
1.4 Motivation 6
1.5 Objectives 6
1.6 Scope of the Project 7
1.7 Literature Review 7
CHAPTER 2 – OVERVIEW OF THE PROJECT 9
2.1 Proposed Solution 9
2.2 Functional Requirements 9
2.3 Non-Functional Requirements 10
2.4 Architecture Components 10
2.5 Module Responsibilities 11
2.6 Compiler Design Concepts Used 11
2.6.1 Lexical Analysis: Tokens, Patterns and Lexemes 11
2.6.2 Context-Free Grammar and Recursive Descent Parsing 12
2.6.3 Abstract Syntax Tree 12
2.6.4 Symbol Table and Scope 13
2.6.5 Type Checking 14
2.6.6 Intermediate Code: Three-Address Code and Quadruples 14
2.6.7 Translation of Control Flow 14
2.6.8 Code Optimization and Basic Blocks 15
2.6.9 Error Detection and Recovery 15
2.7 Methodology 16
2.7.1 Input Processing 16
2.7.2 Lexical Analysis 16
2.7.3 Syntax Analysis 16
2.7.4 Semantic Analysis 16
2.7.5 Intermediate Code Generation 17
2.7.6 Optimization 17
2.7.7 Output and Visualization 17
2.8 Implementation 17
2.8.1 Lexical Analyser 17
2.8.2 Parser 18
2.8.3 Semantic Analyser 19
2.8.4 Code Generator and Optimizer 19

CHAPTER 3 – TECHNOLOGY USED 21
3.1 Technologies Used 21
3.2 Hardware Requirements 21
3.3 Software Requirements 21
3.4 Project Structure 22
CHAPTER 4 – FLOW CHART OF THE PROJECT 23
4.1 Overall System Architecture 23
4.2 Compiler Workflow 23
4.3 Control Flow of the Generated Code 24
4.4 Web Playground Request Flow 25
4.5 Compilation Trace of One Statement 25
CHAPTER 5 – SCREENSHOTS OF THE PROJECT 27
5.1 Web Playground 27
5.2 Output of Each Phase 27
5.3 Command-Line Output 29
5.4 Error Reporting 30
CHAPTER 6 – RESULTS AND CONCLUSION 32
6.1 Test Cases 32
6.2 Sample Outputs 33
Test Case 1 – Valid program with all statement types (TC01) 33
Test Case 2 – Lexical and syntax errors (TC03) 33
Test Case 3 – Semantic errors (TC04) 33
6.3 Performance Evaluation 34
6.4 Challenges Faced 35
6.5 Limitations 36
6.6 Future Scope 36
6.7 Conclusion 37
REFERENCES 39

ABSTRACT

A compiler translates a program through a sequence of phases, but students usually see only the first input and the final error message. The intermediate results that the theory course talks about, such as tokens, syntax trees, symbol tables and three-address code, stay hidden inside production compilers like GCC, which makes the phases hard to connect with real behaviour.

This project presents an end-to-end mini compiler, written in C (C99), for a small C-like language with `int` and `float` variables, arithmetic and relational expressions, `if`/`else`, `while` and `for` statements, nested block scopes and an optional `int main()` with `return`. The compiler is organised as five phases in separate source files: a hand-written lexical analyser, a recursive descent parser that builds an abstract syntax tree with panic-mode error recovery, a semantic analyser that builds a scoped symbol table and performs type checking, an intermediate code generator that emits three-address code as quadruples, and a local optimizer that performs constant propagation, constant folding, branch folding and dead temporary removal. Every phase prints its result to the terminal and to a self-contained HTML report. A small Python server adds a web playground where code typed in the browser is compiled by the same C program and the phases are shown as coloured status lights.

The system was evaluated on 26 October 2026 with 13 functional test cases and 8 automated regression tests. All 13 functional tests and all 8 regression tests passed, and every deliberately inserted error was reported with the correct line number. On the demo program the optimizer reduced the code from 35 to 30 instructions, and a 603-line program compiled in a median of 173 ms. The project shows that a compact compiler of about 1336 lines of C is enough to demonstrate every analysis and synthesis phase taught in the course.

CHAPTER 1 – INTRODUCTION

1.1 Introduction

A **compiler** is a program that reads a program written in a source language and translates it into an equivalent program in a target language, reporting any errors it finds in the source along the way [1]. Compilers make high-level programming possible: the programmer writes readable statements such as `b = a * 2 + 3;` and the compiler produces the low-level instructions the machine executes.

Classical compiler design divides this translation into a series of **phases**, each of which transforms one representation of the program into the next [1], [2]:

- **Lexical analysis** groups characters into tokens such as keywords, identifiers, constants and operators.
- **Syntax analysis** checks that the token sequence follows the grammar of the language and builds a tree.
- **Semantic analysis** checks meaning: declarations, scopes and types, and records names in a symbol table.
- **Intermediate code generation** produces a machine-independent form such as three-address code (TAC).
- **Code optimization** improves the intermediate code without changing what the program computes.
- **Code generation** maps the optimized code to a target machine (outside the scope of this project).

The first three phases form the **analysis** part (the front end) and the remaining phases form the **synthesis** part (the back end). Two structures are shared by all phases: the **symbol table**, which stores information about every name, and the **error handler**, which collects and reports problems found in any phase.

This project implements every phase from lexical analysis to optimization for a small C-like language, and, unlike a production compiler, it shows the output of each phase so that the theory of the course can be seen working on real programs.

1.2 Background

Production compilers such as GCC and Clang are designed for speed and for the quality of the generated machine code. Their internal representations are large, change between versions and are

not printed during a normal compilation. A student who writes `int x = 5.5;` sees at most a warning; the tokens, the tree, the symbol table entry for `x` and the conversion that the compiler inserted all remain invisible.

Compiler courses therefore rely on hand-worked examples on paper: drawing a parse tree, filling a symbol table, writing quadruples for an `if` statement. Tool support exists for single topics (for example, automata simulators or parser generators), but these tools cover one phase at a time and use their own notations, so the student still has to connect the phases mentally. A small compiler whose source code is short enough to read, and whose every phase can be inspected for the same input program, closes this gap.

1.3 Problem Statement

The objective of this project is to develop an end-to-end mini compiler in C that translates programs of a small C-like language through lexical, syntax and semantic analysis, intermediate code generation and optimization. Existing production compilers hide their intermediate representations, and existing educational tools cover only isolated phases. Therefore, there is a need for a system that runs all phases on the same program, detects and reports errors with line numbers in each phase, and visualizes the result of every phase in a form that matches the notation used in the course.

1.4 Motivation

- **Why this problem:** the team found that the phases were easy to describe one at a time but hard to connect; seeing the same statement move from tokens to a tree to quadruples makes the connection obvious.
- **Why build it from scratch:** generator tools such as Lex and Yacc hide the algorithms the course teaches. Writing the lexer and the recursive descent parser by hand shows exactly how tokens are recognised and how grammar rules become functions.
- **Why visualization:** a table or tree that appears next to the source line it came from is easier to understand and to explain in a viva than a description.
- **Who benefits:** students learning compiler design, teaching assistants preparing examples, and the team itself, since each member owns and can explain one phase.

1.5 Objectives

- To design a small C-like source language with a grammar suitable for predictive (LL(1)) parsing.

- To implement a hand-written lexical analyser and a recursive descent parser that builds an abstract syntax tree and recovers from syntax errors.
- To implement semantic analysis with a scoped symbol table and type checking for `int` and `float`.
- To generate three-address code in quadruple form, including translation of `if`/`else`, `while` and `for`.
- To implement local optimizations (constant propagation, constant folding, branch folding, dead temporary removal) and show the code before and after.
- To visualize the output of every phase in the terminal, in an HTML report and in a browser playground, and to evaluate the system using predefined test cases.

1.6 Scope of the Project

- **Target users:** students and faculty of the Compiler Design course.
- **Supported language:** `int` and `float` variables; `+ - * / %` and unary minus; `< > <= >= == !=`; assignment; `if`/`else`; `while`; `for`; `{ }` blocks with their own scope; `//` and `/* */` comments; an optional `int main() { ... return expr; }` wrapper.
- **Functional scope:** lexical, syntax and semantic analysis, intermediate code generation and local optimization, with error detection in the first three phases.
- **Expected usage:** classroom demonstrations, self-study and viva explanations; programs are typically a few dozen lines long.
- **Current limitations:** no target machine code, no user-defined functions other than `main`, no arrays, pointers, strings or input/output, and no logical operators (`&&`, `||`, `!`). These are discussed in Section 6.5.

1.7 Literature Review

Table 1 summarises the systems and publications that influenced the design of the project.

Table 1: Existing Systems and Research

Sr. No.	Paper/System	Year	Approach	Key Finding	Limitation
1	Lex and Yacc (Lesk & Schmidt [5]; Johnson [6])	1975	Generate a lexer from regular expressions and an LALR(1) parser from a grammar	Front ends can be produced automatically from formal specifications	Generated tables hide how tokens and productions are recognised
2	Compilers: Principles, Techniques, and Tools (Aho et al.	2006	Phase-structured compiler with syntax-directed translation to TAC	Standard translation schemes for expressions, control flow and	Textbook examples are worked on paper, not executed

Sr. No.	Paper/System	Year	Approach	Key Finding	Limitation
	[1])			optimization	
3	LLVM (Lattner & Adve [7])	2004	Reusable compiler infrastructure built around an SSA intermediate representation	A well-defined IR lets many front ends share optimizers and back ends	IR and code base are far too large for teaching an introductory course
4	ANTLR (Parr & Quong [8])	1995	Predicated LL(k) parser generator producing recursive descent parsers	Top-down parsers can be generated and still be readable	Requires learning the tool's grammar notation; semantic phases are left to the user
5	JFLAP (Rodger & Finley [16]); Compiler Explorer [9]	2006; 2012	Interactive simulation of automata and parsers; browser view of compiler output	Visual, interactive tools make abstract concepts easier to understand	JFLAP covers single phases; Compiler Explorer shows only final assembly

Table 2 compares the proposed system with production compilers and with compiler generator tools on the features that matter for learning compiler design.

Table 2: Comparative Analysis

Feature	Production compiler (GCC/Clang)	Generator tools (Lex/Yacc, ANTLR)	Proposed system
All phases on one input	Yes, but hidden	Front end only	Yes, all shown
Token list visible	No	Only with extra code	Yes (table)
Syntax tree visible	Only with debug flags	ANTLR only	Yes (collapsible tree)
Symbol table visible	No	No	Yes (with scope and offset)
Three-address code / quadruples	No (internal IR)	No	Yes
Optimization before/after	No	No	Yes, side by side
Errors with line numbers	Yes	Syntax only	Lexical, syntax and semantic
Syntax error recovery	Yes (advanced)	Depends on grammar	Panic mode
Source small enough to read	No (millions of lines)	Generated code is hard to read	Yes (1336 lines of C)

Research gap. Production infrastructures such as GCC and LLVM are complete but opaque, while educational tools such as JFLAP and parser generators are transparent but cover one phase at a time. Neither shows how one input program flows through all phases, how an error in one phase prevents the later phases from running, or how an optimization changes the intermediate code. This project addresses the gap with a single small compiler whose phases are implemented by hand in the notation of the course, and whose output for every phase is shown together for the same program.

CHAPTER 2 – OVERVIEW OF THE PROJECT

2.1 Proposed Solution

The project is a command-line compiler, `minicc`, written in C99, together with two ways of viewing its results. The flow of a compilation is:

- **User interaction:** the user writes a program in a `.mc` file and runs `minicc file.mc`, types a program directly at the keyboard, or types it into the web playground in a browser.
- **Lexical analysis:** `lexer.c` scans the characters and produces an array of tokens with line numbers.
- **Syntax analysis:** `parser.c` checks the tokens against the grammar using recursive descent and builds an abstract syntax tree (AST). On a syntax error it reports the line and skips to the next safe point.
- **Semantic analysis:** `semantic.c` walks the AST, enters every declared variable into a symbol table with its type, scope, size and offset, and checks uses and types.
- **Intermediate code generation:** `codegen.c` walks the typed AST and emits three-address code stored as quadruples, using temporaries `t1, t2, ...` and labels `L1, L2, ...`.
- **Optimization:** the optimizer in `codegen.c` copies the quadruples and simplifies them inside each basic block.
- **Output:** `main.c` prints every phase to the terminal and `report.c` writes `report.html`, a visual page with one section per phase. The web playground shows the same report beside the editor.

Later phases run only when the earlier phases succeeded, because semantic analysis and code generation need a correct tree. The program exits with status 0 when compilation succeeds and 1 when any error was found. The architecture and workflow diagrams are in Chapter 4 (Figure 2 and Figure 3).

2.2 Functional Requirements

Table 3: Functional Requirements

ID	Requirement
FR1	Accept a source program from a file, from the keyboard or from the web playground.
FR2	Recognise keywords, identifiers, integer and float constants, operators and separators, and skip whitespace and comments.
FR3	Report unknown characters, malformed numbers and unterminated comments with line numbers.
FR4	Parse the program according to the grammar and build an abstract syntax tree.

ID	Requirement
FR5	Report syntax errors with the expected and the found token, and continue parsing after an error.
FR6	Build a symbol table with name, type, size, offset, scope level and declaration line.
FR7	Detect undeclared variables, redeclarations, invalid type conversions, invalid operands and misplaced return statements.
FR8	Generate three-address code as quadruples, including control flow for if/else, while and for.
FR9	Optimize the intermediate code and show it before and after optimization.
FR10	Display the output of every phase in the terminal and write a visual HTML report.

2.3 Non-Functional Requirements

- **Correctness:** optimizations must not change program meaning; for example, division by zero is never folded and integer division keeps C semantics.
- **Reliability:** the compiler must never crash or loop forever on bad input; the parser always makes progress during error recovery.
- **Performance:** a typical classroom program (under 50 lines) must compile in well under one second.
- **Portability:** standard C99 only, no external libraries; the HTML report is a single self-contained file.
- **Usability:** error messages name the phase, the line and what was expected; the report works in light and dark browser themes.
- **Maintainability:** one source file per phase with a shared header, so each team member can work on and explain one phase independently.
- **Security:** the web server listens only on 127.0.0.1, limits input size to 100 KB and stops the compiler after 5 seconds.

2.4 Architecture Components

Table 4: Architecture Components

Component	Description
User interface	Command line (`minicc file.mc`) or the web playground (`web/index.html`) served by `server.py`.
Shared definitions	`compiler.h` defines Token, Node (AST), Symbol, Quad and Error and declares the function each phase exports.
Lexical analyser	`lexer.c`: converts the character stream into the global token array.
Parser	`parser.c`: recursive descent parser producing the AST; panic-mode error recovery.
Semantic analyser	`semantic.c`: symbol table with scopes, type checking, annotation of AST node types.
Intermediate code	`codegen.c` (gen_program): emits quadruples from the typed AST.

Component	Description
generator	
Optimizer	`codegen.c` (optimize): local constant propagation and folding, branch folding, dead temporary removal.
Error handler	`error_at()` in `main.c`: records phase, line and message for every error.
Driver and terminal output	`main.c`: reads the source, runs the phases in order, prints every phase.
Visual report	`report.c`: writes `report.html` with a phase status banner, tables and tree.
Web server	`server.py`: receives code from the browser, runs `minicc.exe` and returns the report.

2.5 Module Responsibilities

The work was divided so that each team member owns one phase and can explain it independently. Table 5 lists the modules and their responsibilities.

Table 5: Module Responsibilities

Module	File(s)	Responsibility
Phase 1 – Lexical analysis	lexer.c	Token recognition, keyword lookup, comments, lexical errors
Phase 2 – Syntax analysis	parser.c	Grammar, recursive descent, AST construction, error recovery
Phase 3 – Semantic analysis	semantic.c	Symbol table, scopes, type rules, semantic errors
Phases 4 and 5 – Code generation and optimization	codegen.c	TAC/quadruples, control-flow translation, optimizations
Shared – Driver and visualization	main.c, report.c, compiler.h, server.py, web/index.html	Phase orchestration, terminal output, HTML report, web playground

2.6 Compiler Design Concepts Used

2.6.1 Lexical Analysis: Tokens, Patterns and Lexemes

A **token** is a category of input unit, a **pattern** describes which strings belong to it, and a **lexeme** is an actual string in the source that matches the pattern [1]. The lexer recognises the token categories in Table 6. It reads identifiers with the pattern `[A-Za-z_][A-Za-z0-9_]*` and then checks a keyword table, so keywords need no separate patterns. Operators use the **longest match** rule: `<=` is recognised as one token, not as `<` followed by `=`.

Table 6: Token Categories Recognised by the Lexer

Category	Tokens / pattern	Example lexemes
KEYWORD	int, float, if, else, while, for, return	while
IDENTIFIER	letter or _ followed by letters, digits or _	fact, avg, _tmp
INT_CONST	one or more digits	10, 0
FLOAT_CONST	digits '.' digits	2.5, 3.0

Category	Tokens / pattern	Example lexemes
OPERATOR	+ - * / % = == != < > <= >=	<=
SEPARATOR	( ) { } ; ,	;
END	end of input (added by the lexer)	EOF

2.6.2 Context-Free Grammar and Recursive Descent Parsing

The syntax of the language is defined by the context-free grammar below (EBNF notation: `{ }` means repetition and `[ ]` means optional). The grammar has no left recursion and every choice can be made by looking at the next token, so it is suitable for **predictive (LL(1)) parsing** [1], [14]. The only exception is the program start, where two tokens (`int main`) are examined to decide whether a `main` function is present.

```
program → 'int' 'main' '(' ')' block EOF  |  stmt* EOF
stmt    → decl | assign ';' | if | while | for | block | return
return  → 'return' expr ';'
decl    → ('int' | 'float') ID (',' ID)* ';'
assign  → ID '=' expr
if      → 'if' '(' expr ')' stmt [ 'else' stmt ]
while   → 'while' '(' expr ')' stmt
for     → 'for' '(' assign ';' expr ';' assign ')' stmt
block   → '{' stmt* '}'
expr    → add [ ('<'|'>'|'<='|'>='|'=='|'!=') add ]
add     → mul { ('+'|'-') mul }
mul     → unary { ('*'|'/'|'%') unary }
unary   → '-' unary | primary
primary → INT_CONST | FLOAT_CONST | ID | '(' expr ')'
```

In **recursive descent**, each non-terminal becomes one C function. Operator precedence is encoded by the grammar levels: `expr` (relational) calls `add`, which calls `mul`, which calls `unary`. Left associativity of `+ - * / %` is obtained with loops instead of left-recursive rules, so `a - b - c` is parsed as `(a - b) - c`. Relational operators are non-associative: at most one comparison is allowed per expression.

2.6.3 Abstract Syntax Tree

Instead of a full parse tree with one node per grammar symbol, the parser builds an **abstract syntax tree** that keeps only the meaningful structure: operators, operands and statements. Parentheses and separators do not appear in the tree, because their effect is already captured by its shape. Figure 1 shows the AST for `b = a * 2 + 3;` after semantic analysis has attached a type to every node. Multiplication is deeper in the tree than addition, which is how precedence appears in an AST.

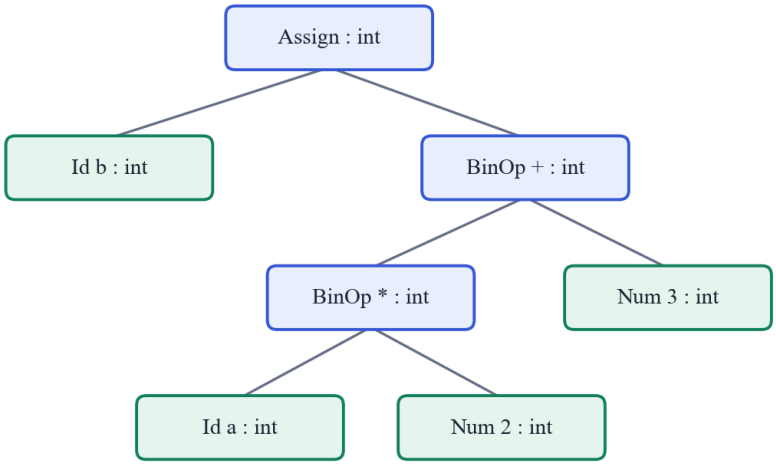


Figure 1: Typed AST for the Statement b = a * 2 + 3;

Table 7: AST Node Kinds

Node	Children	Meaning
Program	list of statements, or one Function	Whole program
Function	body block	int main() { ... }
Block	list of statements	{ ... } with its own scope
Declaration	list of Id nodes	int a, b; / float x;
Assign	target Id, expression	x = expr;
If	condition, then statement, optional else statement	if (c) S1 else S2
While	condition, body	while (c) S
For	init assignment, condition, step assignment, body	for (i; c; s) S
Return	expression	return expr;
BinOp	left, right	+ - * / % and comparisons
Negate	operand	unary minus
Num / Id	none (leaves)	constants and variable names

2.6.4 Symbol Table and Scope

The **symbol table** records every declared name together with its attributes. Each entry stores the name, the type, the storage size (4 bytes for both `int` and `float`), the relative memory **offset** assigned in declaration order, the **scope level** and the declaration line. The table is organised as an array with a parallel *active* flag. Entering a block increases the scope level; leaving it marks that level's entries inactive but keeps them in the table so that they can still be displayed. Lookup searches from the newest entry backwards, so an inner declaration **shadows** an outer one with the same name, as in C. Because the two variables are different, a shadowing variable also receives its own name for the intermediate code: an inner `x` declared after an outer `x` appears in the three-address code as `x.1` (the number is its position in the symbol table). A dot cannot occur in a user identifier, so these names can never clash.

2.6.5 Type Checking

The semantic analyser computes a type for every expression bottom-up and stores it in the AST node. The type rules are summarised in Table 8. When an `int` value is used where a `float` is needed, the type checker allows it (widening), and the code generator later inserts an explicit conversion instruction. The opposite direction would lose information, so it is reported as an error.

Table 8: Type Rules

Construct	Rule	Result / error
Integer / float constant	Contains a decimal point?	float, otherwise int
a op b (+ - * /)	int op int; any float operand	int; float (int operand widened)
a % b	both operands int	int, otherwise error: '%' needs int operands
a relop b	any numeric operands	int (0 or 1)
x = expr	float into int variable	error: type mismatch
x = expr	int into float variable	allowed, code generator inserts (float)
return expr	inside main, expr is int	error if outside main or if expr is float
Use of a name	must be declared and visible	error: undeclared variable

2.6.6 Intermediate Code: Three-Address Code and Quadruples

Three-address code (TAC) is a sequence of simple instructions, each with at most one operator on the right side, such as `t1 = a * 2` [1]. Complex expressions are broken up using compiler-generated temporaries. The project stores TAC as **quadruples**, records with four fields: operator, first argument, second argument and result. Table 9 lists the instruction set.

Table 9: Three-Address Instruction Set

op	arg1	arg2	result	TAC form
+ - * / %	x	y	t	t = x op y
< > <= >= == !=	x	y	t	t = x relop y
uminus	x		t	t = - x
itof	x		t	t = (float) x
=	x		y	y = x
label			L	L:
goto			L	goto L
iffalse	x		L	ifFalse x goto L
return	x			return x

2.6.7 Translation of Control Flow

Boolean conditions are evaluated into a temporary (0 or 1), and the statements are translated with labels and conditional jumps following the standard schemes [1]:

```
if (c) S1 else S2          while (c) S              for (i; c; s) S
    t = c                  L1: t = c                    i
    ifFalse t goto L1          ifFalse t goto L2    L1: t = c
    S1                         S                        ifFalse t goto L2
    goto L2                    goto L1                  S
L1: S2                     L2:                          s
L2:                                                     goto L1
                                                    L2:
```

A `for` statement is translated as its initialisation followed by a `while` loop whose body is the loop body followed by the step. Fresh labels are taken from a counter, so nested statements never clash.

2.6.8 Code Optimization and Basic Blocks

A **basic block** is a maximal sequence of instructions that is entered only at the first instruction and left only at the last [1]. In the generated code, every label starts a new block, because control can reach a label from more than one place (Figure 4 in Chapter 4 shows the blocks of a `for` loop).

The optimizer applies four **local** transformations, summarised in Table 10. Constant propagation and folding follow the classical data-flow idea of tracking which names hold known constant values [15], restricted to a single basic block: at every label the table of known values is cleared.

Table 10: Optimizations Implemented

Optimization	Example (before → after)	Condition
Constant propagation	a = 5; t1 = a * 2 → t1 = 5 * 2	a has a known constant value in this block
Constant folding	t1 = 5 * 2 → t1 = 10	both operands constant; division by zero never folded
Branch folding	ifFalse 1 goto L → (removed); ifFalse 0 goto L → goto L	condition is a constant
Dead temporary removal	t1 = 10 with no later use of t1 → (removed)	temporary is never read

2.6.9 Error Detection and Recovery

Each phase detects the errors it is able to see. Table 11 lists them. The parser uses **panic-mode recovery** [1]: after the first error in a statement it stops reporting, skips tokens until it reaches a synchronising token (`;` or `}`) and resumes with the next statement. This lets one run report several independent errors while avoiding long cascades of messages caused by a single mistake.

Table 11: Errors Detected by Each Phase

Phase	Error	Example message
Lexical	Unknown character	line 5: unknown character '@'
Lexical	Malformed number	line 2: malformed number '12abc'
Lexical	Unterminated comment	line 3: unterminated comment
Syntax	Missing or unexpected token	line 6: expected ')' but found ';'
Syntax	Code after main()	line 6: expected end of program after main() but found 'int'

Phase	Error	Example message
Semantic	Undeclared variable	line 9: undeclared variable 'z'
Semantic	Redeclaration in the same scope	line 4: redeclaration of 'x' (first declared on line 2)
Semantic	Type mismatch	line 7: type mismatch: cannot assign float to int variable 'x'
Semantic	Invalid operands	line 8: operator '%' needs int operands, got float % int
Semantic	Misplaced or wrong return	line 4: 'return' used outside of main()
Semantic	Reserved name	't1' is reserved for compiler temporaries

2.7 Methodology

2.7.1 Input Processing

The driver reads the whole source program into a single character buffer. With a file argument it opens the file in text mode; without one it reads standard input until end-of-file, which allows a program to be typed interactively. The web server writes the code received from the browser into a temporary file and runs the compiler on that file, so all three entry points use the same path from this point on.

2.7.2 Lexical Analysis

The lexer walks the buffer once with a pointer and a line counter. Whitespace is skipped and newlines increase the line counter. Comments are removed here so that the parser never sees them. For each token the lexer records its kind, its lexeme and its line, appending it to a dynamically growing array. Lexical errors are recorded and the offending characters are skipped, so the scan always reaches the end of the input. An EOF token is added at the end so that the parser never reads past the array.

2.7.3 Syntax Analysis

The parser keeps an index into the token array. Each grammar function inspects the current token to choose a production, consumes the tokens it expects with `expect()`, and returns a newly allocated AST node. When a token does not match, `syntax_error()` records the line, the expected item and the found lexeme, and sets the panic flag. The statement-list loop then synchronises and clears the flag, and a safety check advances the position if a statement consumed nothing, which guarantees termination on any input.

2.7.4 Semantic Analysis

The semantic analyser performs a depth-first walk over the AST. Declarations insert symbols; blocks open and close scopes; expressions are checked bottom-up and their types written into the nodes. When a sub-expression already has an error type, the parent does not report a second error,

which avoids cascades such as one undeclared variable producing several type errors.

2.7.5 Intermediate Code Generation

Code generation is a second depth-first walk over the typed AST. For an expression, the generator returns the name of the place holding its value: a variable, a constant or a new temporary. For statements it applies the translation schemes of Section 2.6.7. Where an `int` operand meets a `float` operand, or an `int` value is assigned to a `float` variable, an `itof` quadruple is emitted first.

2.7.6 Optimization

The optimizer reads the quadruples in order and writes an improved copy. It keeps a small table mapping names to constant values. For each instruction it first replaces operands with known constants (propagation), then evaluates the instruction if all operands are constant (folding), then updates the table: a copy of a constant records the value, any other definition forgets it. Labels clear the table because they start a new basic block. A final pass removes assignments to temporaries that are no longer read anywhere.

2.7.7 Output and Visualization

The driver prints a section per phase to the terminal: the token table, an ASCII drawing of the AST with types, the symbol table, the quadruples with their TAC form, the optimized code and the error list. `report.c` writes the same information to `report.html`, adding a pipeline banner that marks each phase as passed, failed or skipped, the source with error lines highlighted, and a collapsible tree. The web playground reads the banner's status classes to colour its own phase lights and shows the report beside the editor.

2.8 Implementation

2.8.1 Lexical Analyser

The core of the lexer is a loop over the characters. The excerpt below shows how operators are recognised: two-character operators are tested first so that the longest match wins, and any character that matches no pattern is reported and skipped.

```
/* two-character operators first */
TokenKind kind;
int len = 2;
if      (p[0] == '=' && p[1] == '=') kind = TK_EQ;
else if (p[0] == '!' && p[1] == '=') kind = TK_NE;
else if (p[0] == '<' && p[1] == '=') kind = TK_LE;
else if (p[0] == '>' && p[1] == '=') kind = TK_GE;
else {
    len = 1;
    switch (*p) {
    case '+': kind = TK_PLUS; break;
    case '-': kind = TK_MINUS; break;
    case '*': kind = TK_STAR; break;
    case '/': kind = TK_SLASH; break;
    case '%': kind = TK_PERCENT; break;
    case '=': kind = TK_ASSIGN; break;
    case '<': kind = TK_LT; break;
    case '>': kind = TK_GT; break;
    case '(': kind = TK_LPAREN; break;
    case ')': kind = TK_RPAREN; break;
    case '{': kind = TK_LBRACE; break;
    case '}': kind = TK_RBRACE; break;
    case ';': kind = TK_SEMI; break;
    case ',': kind = TK_COMMA; break;
    default:
        error_at("Lexical", line, "unknown character '%c'", *p);
        p++;
        continue;
    }
}
add(kind, p, len, line);
p += len;
```

2.8.2 Parser

Each grammar rule is a function. The function for `add → mul { ('+' | '-') mul }` shows how a loop gives left associativity: each new operator node takes the tree built so far as its left child.

```
static Node *add(void)
{
    Node *n = mul();
    while (CUR.kind == TK_PLUS || CUR.kind == TK_MINUS) {
        Token op = CUR;
        advance();
        n = binop(op, n, mul());
    }
    return n;
}
```

Error recovery is implemented in the statement-list function. If a statement set the panic flag, the parser skips to the next `;` or `}` unless the statement already ended at one, and it always advances at least one token, so parsing finishes on any input.

```
static Node *stmt_list(TokenKind end)
{
    Node *first = NULL, **tail = &first;
    while (CUR.kind != end && CUR.kind != TK_EOF) {
        int start = pos;
        Node *s = stmt();
        if (panic) {   /* skip to a safe point, unless the statement already ended at one */
            TokenKind last = pos > start ? tokens[pos - 1].kind : TK_EOF;
            if (last != TK_SEMI && last != TK_RBRACE) {
                while (CUR.kind != TK_SEMI && CUR.kind != TK_RBRACE && CUR.kind != TK_EOF) advance();
                accept(TK_SEMI);
            }
            panic = 0;
        }
        if (pos == start) advance();   /* always make progress */
        if (s) { *tail = s; tail = &s->next; }
    }
    return first;
}
```

2.8.3 Semantic Analyser

The `declare` function shows the symbol table insertion: a redeclaration is reported only when the existing symbol is in the *same* scope, which is what allows shadowing in inner blocks.

```
static void declare(Node *id, Type type)
{
    Symbol *old = lookup(id->text);
    if (old && old->scope == scope) {
        error_at("Semantic", id->line, "redeclaration of '%s' (first declared on line %d)",
                 id->text, old->line);
        return;
    }
    if (is_temp_name(id->text)) {
        error_at("Semantic", id->line, "'%s' is reserved for compiler temporaries", id->text);
        return;
    }
    if (nsyms == MAX_SYMS) {
        error_at("Semantic", id->line, "too many variables (max %d)", MAX_SYMS);
        return;
    }
    Symbol *s = &symtab[nsyms];
    strcpy(s->name, id->text);
    /* a later variable with an existing name (e.g. shadowing in a block) gets its own TAC name */
    strcpy(s->tac, id->text);
    for (int i = 0; i < nsyms; i++)
        if (!strcmp(symtab[i].name, id->text)) sprintf(s->tac, "%s.%d", id->text, nsyms);
    s->type = type;
    s->size = 4;
    s->offset = offset;
    s->scope = scope;
    s->line = id->line;
    active[nsyms++] = 1;
    offset += s->size;
    id->type = type;
    id->sym = s;
}
```

2.8.4 Code Generator and Optimizer

The translation of a `while` statement follows the scheme of Section 2.6.7 directly: two fresh labels, the condition, a conditional jump out of the loop, the body and a jump back.

```
case N_WHILE:
    new_label(l1);
    new_label(l2);
    emit("label", "", "", l1);
    gen_expr(n->kid[0], v);
    emit("iffalse", v, "", l2);
    gen_stmt(n->kid[1]);
    emit("goto", "", "", l1);
    emit("label", "", "", l2);
    break;
```

Inside the optimizer, folding is applied after propagation has replaced operands with constants. The helper `fold()` refuses to fold division or modulus by zero and uses integer arithmetic unless one operand is a float constant, so the folded result is exactly what the program would compute at run time.

```
/* constant folding */
if (!strcmp(q.op, "itof") && is_const(q.arg1)) {
    format_float(atof(q.arg1), folded);
    make_copy(&q, folded);
} else if (!strcmp(q.op, "uminus") && is_const(q.arg1)) {
    if (fold("-", is_float_const(q.arg1) ? "0.0" : "0", q.arg1, folded)) make_copy(&q, folded);
} else if (q.arg2[0] && is_const(q.arg1) && is_const(q.arg2)) {
    if (fold(q.op, q.arg1, q.arg2, folded)) make_copy(&q, folded);
}

if (!strcmp(q.op, "=") && is_const(q.arg1)) remember(q.res, q.arg1);
else forget(q.res);
opt[nopt++] = q;
```

CHAPTER 3 – TECHNOLOGY USED

3.1 Technologies Used

Table 12 lists every technology used in the project and what it was used for. The compiler itself uses only the C standard library; everything else is optional tooling around it.

Table 12: Technologies Used

Technology	Version	Used for
C programming language	C99 (ISO/IEC 9899:1999) [11]	The compiler: all five phases, terminal output and the HTML report
GCC (MinGW)	6.3.0	Building `minicc.exe` with `-std=c99 -Wall -Wextra` [10]
Python	3.12.10	Local web server for the playground (standard library `http.server` and `subprocess` only) [12]
HTML5, CSS, JavaScript	–	The generated `report.html` and the browser playground
Windows batch scripts	–	`build.bat` (build), `test.bat` (regression tests), `web.bat` (playground)
Windows 11 and PowerShell	–	Development and testing environment
Google Chrome	–	Viewing the report and the playground

3.2 Hardware Requirements

Table 13: Hardware Requirements

Component	Requirement
Processor	Any x86-64 processor (tested on AMD Ryzen 5 5600H)
RAM	512 MB free is sufficient; the compiler uses fixed-size tables (tested on 8 GB system)
Storage	Under 1 MB for the source code and executable (minicc.exe is about 136 KB)
GPU	Not required
Internet Connection	Not required; the report and playground run entirely on the local machine

3.3 Software Requirements

Table 14: Software Requirements

Software/Technology	Version/Specification
Operating System	Windows 11 (tested); the C code is portable C99 and also builds on Linux
Programming Language	C (C99 standard) for the compiler; Python 3 for the optional web server
Compiler for building	GCC 6.3.0 (MinGW) with -std=c99 -Wall -Wextra
Editor	Any text editor

Software/Technology	Version/Specification
Web playground	Python 3 standard library; no packages to install
Browser	Any modern browser (tested in Google Chrome)
External libraries	None; only the C standard library

3.4 Project Structure

```
202307020045_CompilerDesignMiniProject/
├── src/
│   ├── compiler.h      shared structures and phase interfaces
│   ├── lexer.c         Phase 1: lexical analysis
│   ├── parser.c        Phase 2: recursive descent parser, AST
│   ├── semantic.c      Phase 3: symbol table and type checking
│   ├── codegen.c       Phases 4-5: quadruples and optimizer
│   ├── report.c        HTML phase report
│   └── main.c          driver, terminal output, error list
├── web/index.html      browser playground
├── server.py           local web server for the playground
├── examples/           demo.mc, main.mc, errors.mc, semantic.mc
├── tests/              extra test programs and expected outputs
├── report/             this report
├── build.bat           build with gcc
├── test.bat            regression tests
├── web.bat             start the playground
└── README.md
```

Table 15: Source Code Size by Module

File	Lines	Purpose
lexer.c	138	Lexical analysis
parser.c	297	Syntax analysis and AST
semantic.c	166	Semantic analysis
codegen.c	315	Intermediate code and optimization
main.c	123	Driver, errors, terminal output
report.c	195	HTML report
compiler.h	102	Shared definitions
Total (C)	**1336**	
server.py / web/index.html	62 / 171	Web playground (optional)

Build with `build.bat`, run with `.\minicc examples\demo.mc`, test with `test.bat`, and start the playground with `web.bat`.

GitHub Repository: https://github.com/Hetsharma5/CompilerDesignMiniProject.git

CHAPTER 4 – FLOW CHART OF THE PROJECT

4.1 Overall System Architecture

Figure 2 shows the architecture as implemented. Both user interfaces feed source code into the same compiler executable. Inside the compiler, the five phases are connected in a pipeline; each phase consumes the data structure produced by the previous one. All phases share the error list and the data structure definitions in `compiler.h`. The two output modules read the results of all phases at the end of the run.

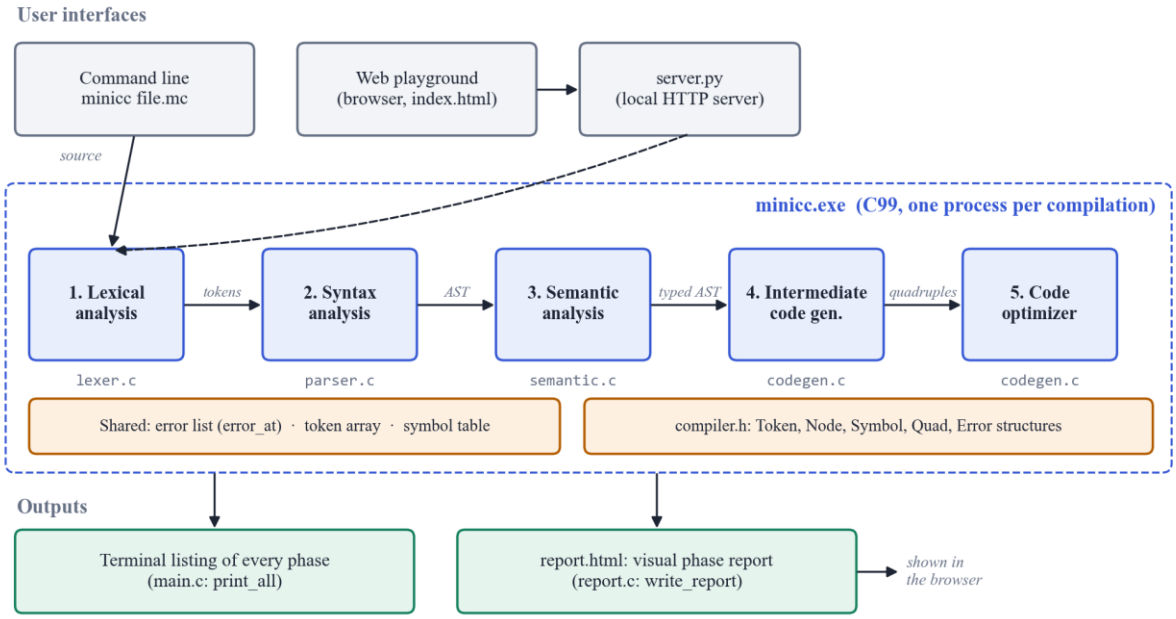


Figure 2: Overall System Architecture

4.2 Compiler Workflow

Figure 3 is the flow chart of one compilation, including the decisions that skip the later phases when errors are found. In words:

Step 1: The source program is read completely into memory (from a file, the keyboard or the web server).

Step 2: The lexer scans it once from left to right and produces the token array, ending with an EOF token.

Step 3: The parser reads the tokens with one token of lookahead and builds the AST, reporting syntax errors.

Step 4: If no lexical or syntax error occurred, the semantic analyser walks the AST, fills the symbol table and annotates every expression with its type.

Step 5: If no error occurred at all, the code generator walks the typed AST and emits quadruples.

Step 6: The optimizer produces an improved copy of the quadruples.

Step 7: The driver prints every phase, writes `report.html` and exits with status 0 or 1.

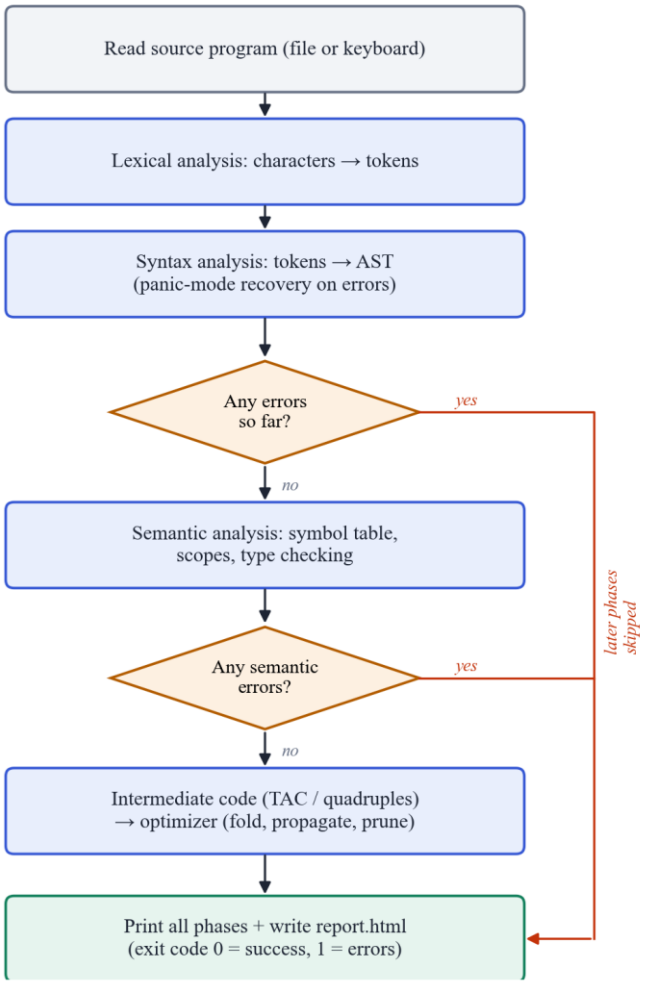


Figure 3: Flow Chart of the Compiler with Error Handling

4.3 Control Flow of the Generated Code

Figure 4 shows the basic blocks of the three-address code generated for the `for` loop in `examples/main.mc`, and the jumps between them. The conditional jump leaves the loop; the `goto` at the end of the body is the back edge.

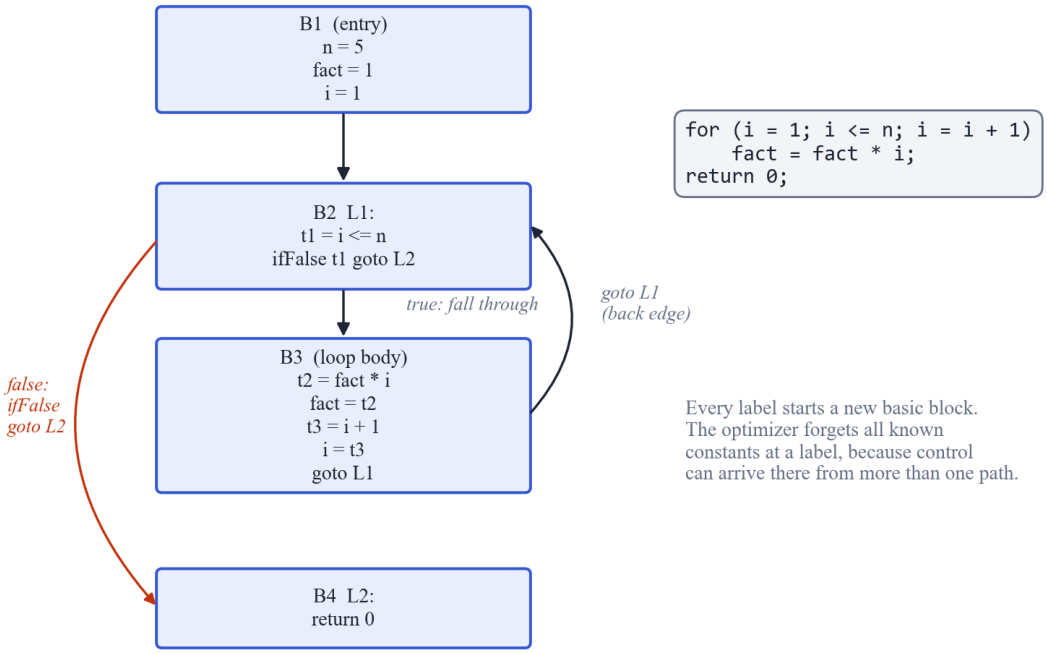


Figure 4: Basic Blocks and Control Flow of a for Loop

4.4 Web Playground Request Flow

Figure 5 shows what happens when Compile is pressed in the browser: the code is sent to `server.py`, which runs the unchanged C compiler in a temporary folder and returns the generated report.

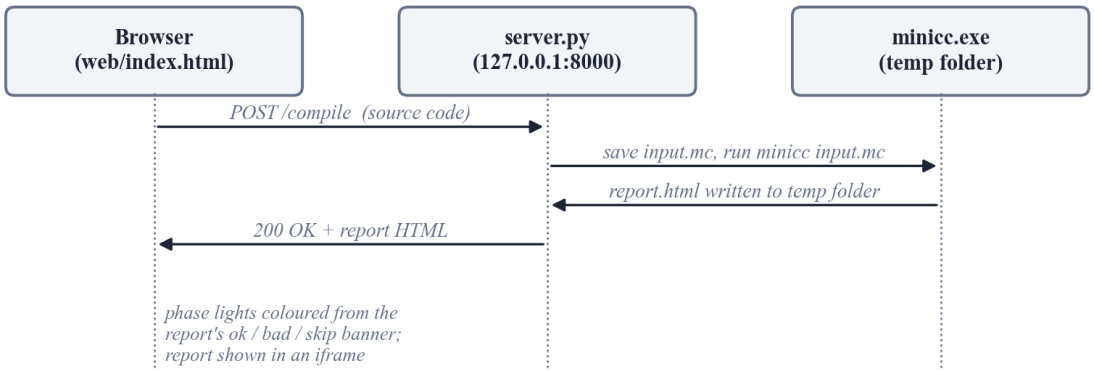


Figure 5: Web Playground Request Flow

4.5 Compilation Trace of One Statement

The trace below follows the single statement `b = a * 2 + 3;` (line 6 of the demo program, with `a = 5` on the line before) through every phase:

```
Source            b = a * 2 + 3;
      ↓
Lexical analysis  IDENTIFIER(b) OPERATOR(=) IDENTIFIER(a) OPERATOR(*)
                  INT_CONST(2) OPERATOR(+) INT_CONST(3) SEPARATOR(;)
      ↓
Syntax analysis   Assign( Id b, BinOp +( BinOp *( Id a, Num 2 ), Num 3 ) )
      ↓
Semantic          b: int (scope 0, offset 4)   a * 2 : int   ... + 3 : int
      ↓
Intermediate      t1 = a * 2
code              t2 = t1 + 3
                  b = t2
      ↓
Optimization      a = 5 is known  →  t1 = 5 * 2 = 10  →  t2 = 10 + 3 = 13
                  →  b = 13   (t1, t2 no longer used: removed)
```

CHAPTER 5 – SCREENSHOTS OF THE PROJECT

5.1 Web Playground

The **web playground** (Figure 6) has an editor with line numbers, a list of sample programs, a Compile button (also Ctrl+Enter) and five phase lights.

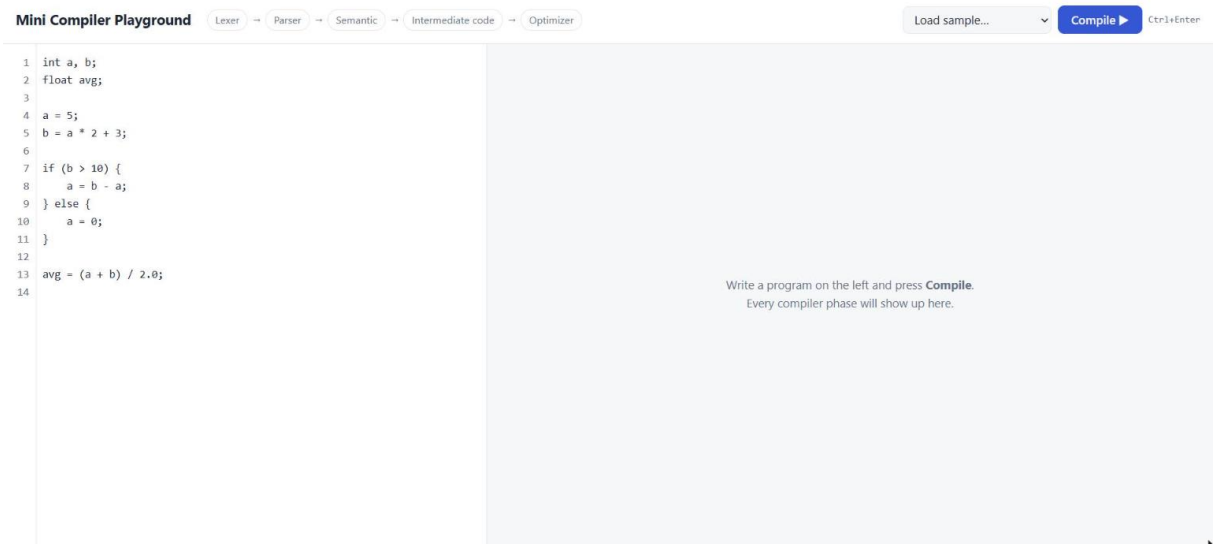


Figure 6: Web Playground – Home Page with the Editor

After compilation, each phase light turns green (passed), red (errors) or grey (skipped), and the full report appears on the right (Figure 7).

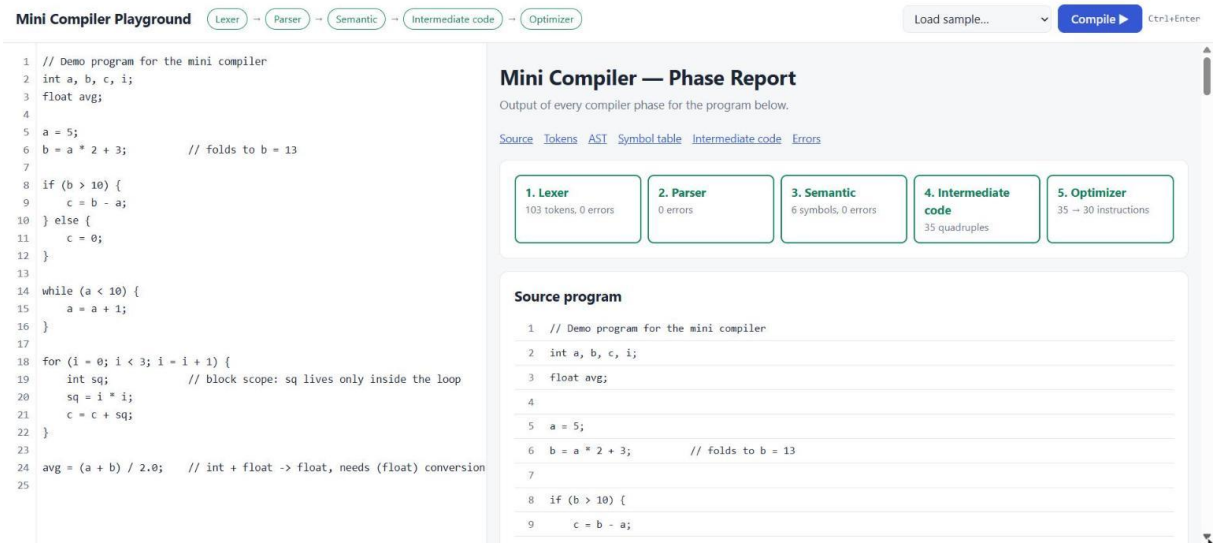


Figure 7: Compiling the Demo Program – All Five Phases Pass

5.2 Output of Each Phase

Figure 8 to Figure 11 show the report sections for the demo program `examples/demo.mc`.

Phase 1 · Lexical analysis — tokens

#	Line	Category	Lexeme
0	2	KEYWORD	int
1	2	IDENTIFIER	a
2	2	SEPARATOR	,
3	2	IDENTIFIER	b
4	2	SEPARATOR	,
5	2	IDENTIFIER	c
6	2	SEPARATOR	,
7	2	IDENTIFIER	i
8	2	SEPARATOR	;
9	3	KEYWORD	float
10	3	IDENTIFIER	avg
11	3	SEPARATOR	;
12	5	IDENTIFIER	a
13	5	OPERATOR	=
14	5	INT_CONST	5
15	5	SEPARATOR	;
16	6	IDENTIFIER	b
17	6	OPERATOR	=

Figure 8: Phase 1 Output – Token Table with Line Numbers

Phase 2 · Syntax analysis — abstract syntax tree

Click a node to collapse it. Grey tags show the type found in phase 3.

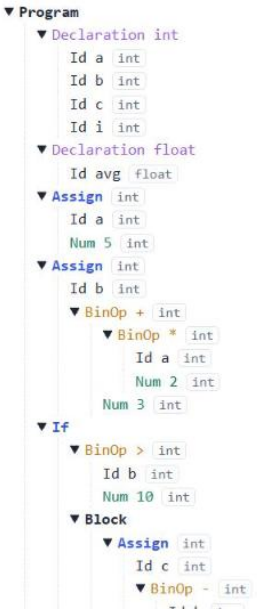


Figure 9: Phase 2 Output – Collapsible AST Annotated with Types

Phase 3 · Semantic analysis — symbol table

Name	Type	Size	Offset	Scope	Declared on line
a	int	4	0	0	2
b	int	4	4	0	2
c	int	4	8	0	2
i	int	4	12	0	2
avg	float	4	16	0	3
sq	int	4	20	1	19

Figure 10: Phase 3 Output – Symbol Table with Scope and Offset

Phase 4 & 5 · Intermediate code and optimization

Before optimization

#	Three address code	op	arg1	arg2	result
0	a = 5	=	5		a
1	t1 = a * 2	*	a	2	t1
2	t2 = t1 + 3	+	t1	3	t2
3	b = t2	=	t2		b
4	t3 = b > 10	>	b	10	t3
5	ifFalse t3 goto L1	iffalse	t3		L1
6	t4 = b - a	-	b	a	t4
7	c = t4	=	t4		c
8	goto L2	goto			L2
9	L1:	label			L1
10	c = 0	=	0		c
11	L2:	label			L2
12	L3:	label			L3
13	t5 = a < 10	<	a	10	t5
14	ifFalse t5 goto L4	iffalse	t5		L4
15	t6 = a + 1	+	a	1	t6
16	a = t6	=	t6		a
17	goto L3	goto			L3

After optimization

#	Three address code	op	arg1	arg2	result
0	a = 5	=	5		a
1	b = 13	=	13		b
2	c = 8	=	8		c
3	goto L2	goto			L2
4	L1:	label			L1
5	c = 0	=	0		c
6	L2:	label			L2
7	L3:	label			L3
8	t5 = a < 10	<	a	10	t5
9	ifFalse t5 goto L4	iffalse	t5		L4
10	t6 = a + 1	+	a	1	t6
11	a = t6	=	t6		a
12	goto L3	goto			L3
13	L4:	label			L4
14	i = 0	=	0		i
15	L5:	label			L5
16	t7 = i < 3	<	i	3	t7
17	ifFalse t7 goto L6	iffalse	t7		L6

Figure 11: Phases 4 and 5 – Quadruples Before and After Optimization

5.3 Command-Line Output

The same information is printed in the terminal (Figure 12), so the compiler can also be demonstrated without a browser.

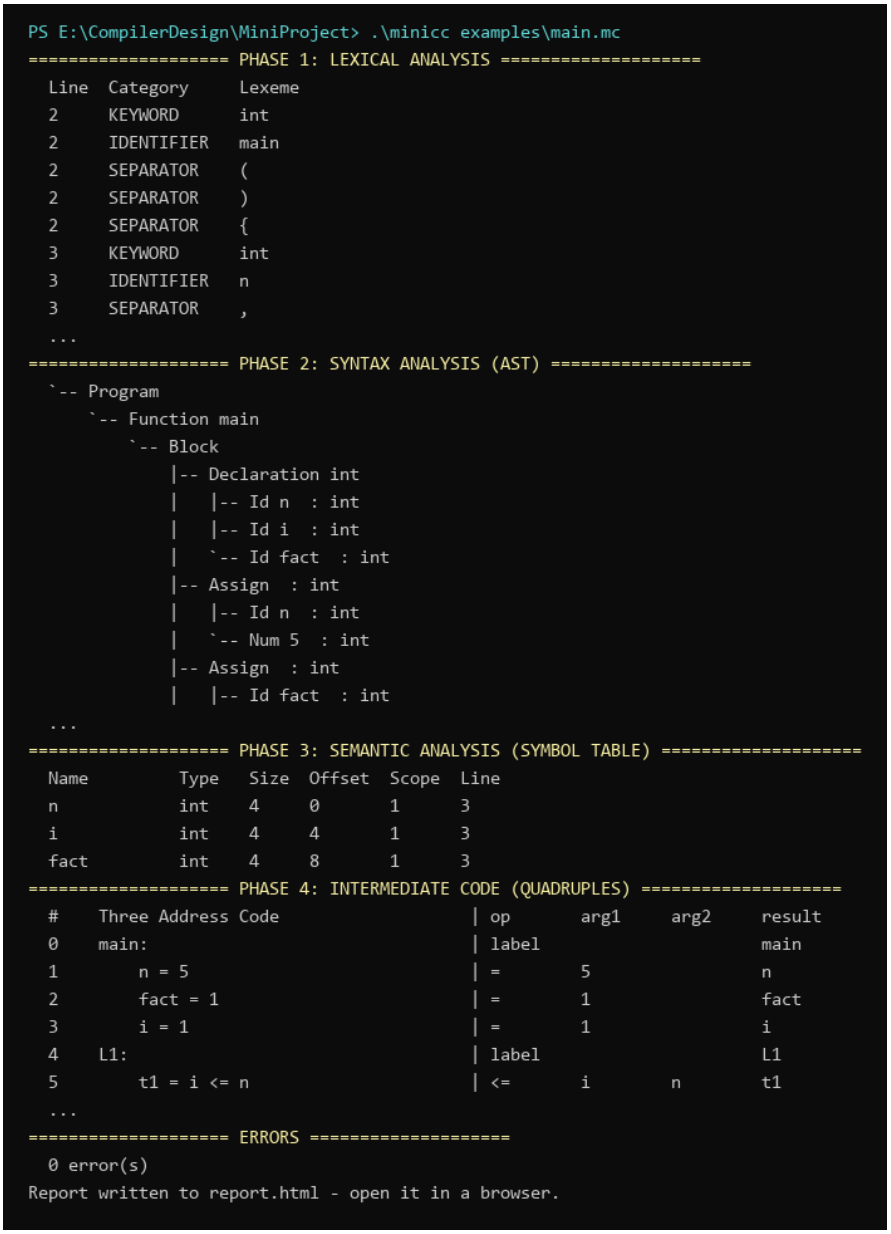


Figure 12: Command-Line Output of minicc for examples/main.mc (excerpt)

5.4 Error Reporting

Figure 13 shows a program with lexical and syntax errors: the failed phases are red, the later phases are skipped and the error lines are highlighted. Figure 14 shows semantic errors, and Figure 15 the error list of the HTML report.

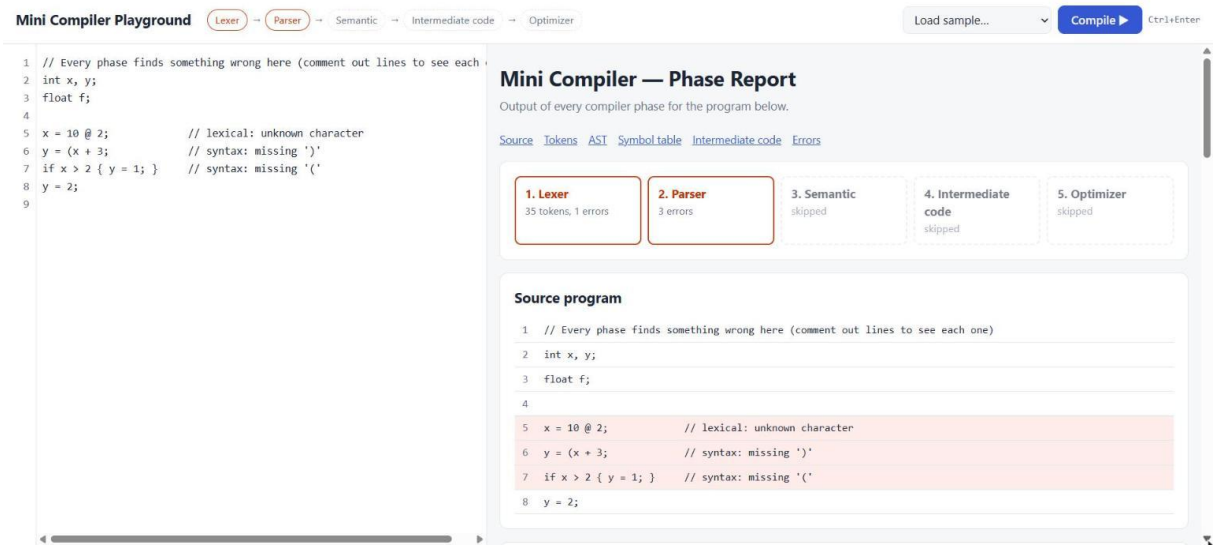


Figure 13: Lexical and Syntax Errors in the Playground (failed phases in red, error lines highlighted)

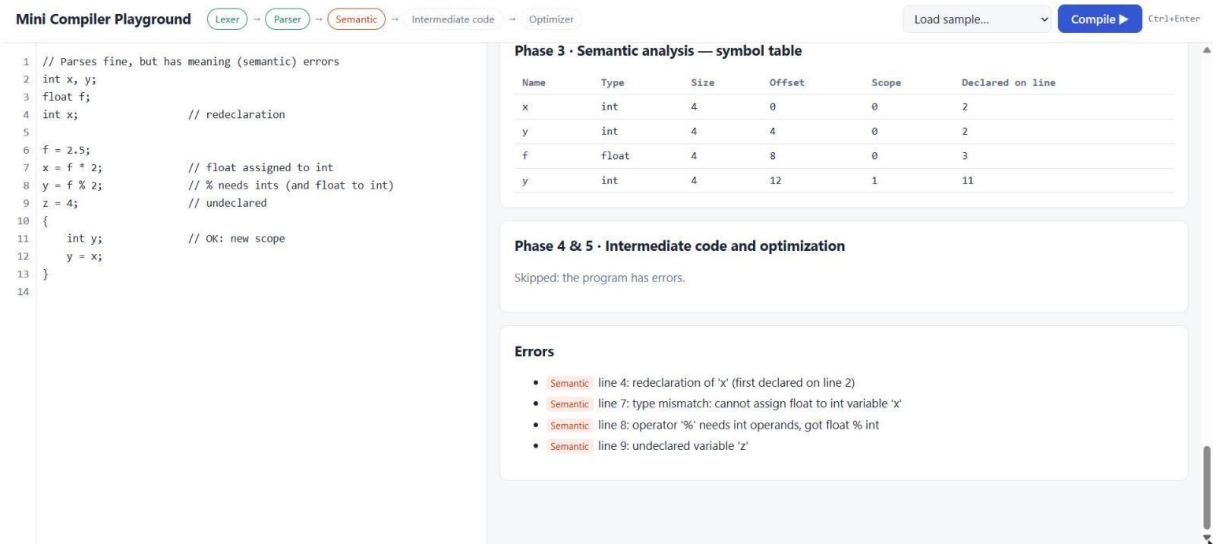


Figure 14: Semantic Errors Reported with Line Numbers

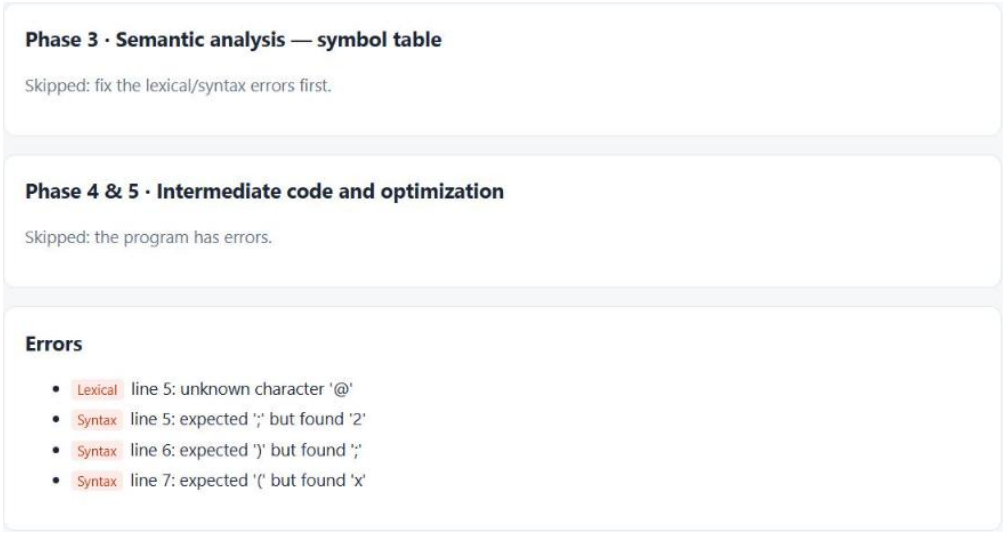


Figure 15: Error List in the HTML Report, Grouped by Phase

CHAPTER 6 – RESULTS AND CONCLUSION

6.1 Test Cases

Table 16 lists the functional test cases, all executed on 26 October 2026 with the final build. The *Actual Output* column is copied from the compiler's output. A test passes when the compiler accepts valid programs with the expected code and rejects invalid programs with the expected error messages and line numbers.

Table 16: Test Cases

Test Case ID	Input	Expected Output	Actual Output	Status
TC01	Valid program with declarations, if-else, while, for and int/float mixing (`demo.mc`)	0 errors; b folded to 13; if-branch decided at compile time	0 errors; 35 → 30 instructions	Pass
TC02	Program wrapped in int main() with return 0 (`main.mc`)	0 errors; main: label and return 0 in TAC	0 errors; 14 → 14 instructions	Pass
TC03	Lexical and syntax errors in one file (`errors.mc`)	unknown '@' (line 5); missing ')' (line 6); missing '(' (line 7)	Lexical line 5: unknown character '@'; Syntax line 5: expected ';' but found '2'; Syntax line 6: expected ')' but found ';'; Syntax line 7: expected '(' but found 'x'	Pass
TC04	Semantic errors: redeclaration, type mismatch, % on float, undeclared (`semantic.mc`)	4 semantic errors on lines 4, 7, 8, 9	Semantic line 4: redeclaration of 'x' (first declared on line 2); Semantic line 7: type mismatch: cannot assign float to int variable 'x'; Semantic line 8: operator '%' needs int operands, got float % int; Semantic line 9: undeclared variable 'z'	Pass
TC05	Optimizer edge cases (`optimizer.mc`)	7/0 not folded; k = 4; f = 3.0 then 4.5; while(0) becomes goto	0 errors; 17 → 11 instructions; total = 7, t1 = 7 / 0, z = t1, k = 4, f = 3.0, f = 4.5, L1:, goto L2	Pass
TC06	Operator precedence and unary minus (`precedence.mc`)	a = 13 and b = 14 after folding	0 errors; 8 → 2 instructions; a = 13, b = 14	Pass
TC07	Nested block scope (shadowing) (`scope.mc`)	0 errors; inner x renamed x.1 in TAC; outer x = 2 after optimization	0 errors; 4 → 3 instructions; x = 1, x.1 = 2.5, x = 2	Pass
TC08	Malformed number (`badnum.mc`)	malformed number error on line 2	Lexical line 2: malformed number '12abc'; Syntax line 2: expected an expression but found ';'	Pass
TC09	Unterminated block comment (`comment.mc`)	unterminated comment error on line 3	Lexical line 3: unterminated comment	Pass
TC10	return outside main() (`return_outside.mc`)	return outside main reported	Semantic line 4: 'return' used outside of main()	Pass
TC11	main() returning float (`main_errors.mc`)	float return from main reported	Semantic line 4: type mismatch: main() must return int, not float	Pass
TC12	Empty program (`empty.mc`)	0 errors, 0 instructions	0 errors; 0 → 0 instructions	Pass
TC13	Standard C Hello World (unsupported features) (`hello.mc`)	rejected: '#', "" and other unsupported characters reported	Lexical line 1: unknown character '#'; Lexical line 1: unknown character '.'; Lexical line 4: unknown character ""; Lexical line 4: unknown character '!'; … (8 errors)	Pass

In addition, `test.bat` runs 8 programs and compares the complete output of the compiler with

saved expected output files (golden testing). This catches any unintended change in any phase.

Table 17: Automated Regression Tests (test.bat)

Test	Program	Result
1	examples/demo.mc	PASS
2	examples/errors.mc	PASS
3	examples/semantic.mc	PASS
4	examples/main.mc	PASS
5	tests/optimizer.mc	PASS
6	tests/return_outside.mc	PASS
7	tests/main_errors.mc	PASS
8	tests/scope.mc	PASS

6.2 Sample Outputs

Test Case 1 – Valid program with all statement types (TC01)

Input: `examples/demo.mc` (25 lines: declarations, `if`/`else`, `while`, `for` with a block-scoped variable, and an `int`/`float` expression).

Compiler action: all five phases run. The lexer produced 103 tokens, the symbol table contains 6 symbols (`sq` at scope level 1), the code generator emitted 35 quadruples and the optimizer reduced them to 30.

Output: `b = a * 2 + 3` became `b = 13`; the condition `b > 10` was evaluated at compile time, so its `ifFalse` was removed and `c = b - a` became `c = 8`; `avg = (a + b) / 2.0` received a `(float)` conversion. Screenshots: Figure 7 to Figure 11.

Test Case 2 – Lexical and syntax errors (TC03)

Input: `examples/errors.mc`, containing an unknown character `@`, a missing `)` and an `if` without `(`.

Compiler action: the lexer reports the `@` and skips it; the parser reports each syntax error and recovers at the next `;`, so all three mistakes are found in one run. Semantic analysis and code generation are skipped.

Output: four messages – the three intended errors plus one secondary message (`expected ';' but found '2'`) caused by the skipped `@` (Figure 13).

Test Case 3 – Semantic errors (TC04)

Input: `examples/semantic.mc`, which is syntactically correct but redeclares `x`, assigns a float to an int, uses `%` on a float and assigns to an undeclared `z`. It also declares a second `y` inside a block, which is legal.

Compiler action: the semantic analyser reports exactly four errors and accepts the inner `y`, which appears in the symbol table at scope level 1 (Figure 14).

6.3 Performance Evaluation

All values in this section were measured on 26 October 2026 on the development machine (AMD Ryzen 5 5600H, Windows 11). Times were measured from Python with `time.perf_counter()` around a complete run of `minicc.exe`, so they include process start-up and writing `report.html`; each value is the median of 20 runs.

Table 18: Performance Evaluation

Metric	Evaluation Method	Result
Test case success rate	Passed functional tests / total	13 / 13 (100%)
Regression test success rate	Passed golden-output tests / total	8 / 8 (100%)
Error detection rate	Deliberately inserted errors reported with correct line / inserted errors	11 / 11 (100%)
Secondary (cascade) messages	Extra messages caused by an earlier error, over TC03 and TC08	2
Code reduction – demo program	Instructions after / before optimization	35 → 30 (14.3% fewer)
Code reduction – optimizer test	Instructions after / before optimization	17 → 11 (35.3% fewer)
Average compile time (test programs)	Median wall-clock time per program, TC01–TC13	15.5–24.2 ms (mean 18.9 ms)
Compile time, 603-line program	Median of 20 runs	172.9 ms

Table 19 shows how compile time grows with program size, using generated programs made of repeated statements of the form `x = x + i * k;`. Because `x` and `i` start with constant values, the optimizer folds every statement to a single constant assignment, which explains the large reduction in instructions.

Table 19: Compile Time and Optimization by Program Size

Source lines	Tokens	Quadruples	After optimization	Median time (ms)
53	414	152	52	38.8
153	1214	452	152	59.5
303	2414	902	302	104.3
603	4814	1802	602	172.9

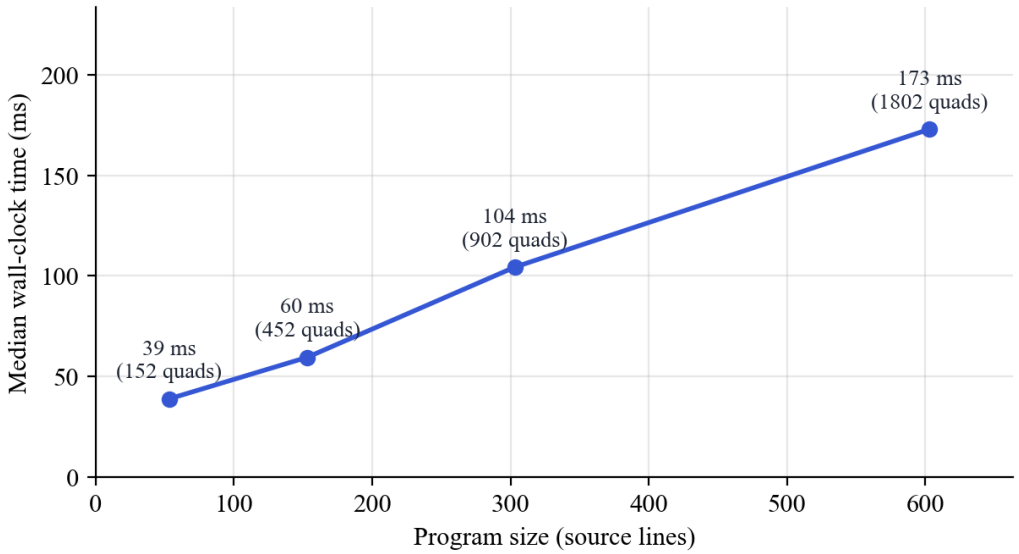


Figure 16: Compile Time versus Program Size

The time grows roughly linearly with program size, as expected for single-pass lexing and parsing and linear tree walks. Even an empty program takes 20 ms, which is the fixed cost of starting the process and writing the report; it dominates for classroom-sized programs. The dead-temporary pass compares every temporary with every instruction, which is quadratic in the worst case; it is not noticeable at these sizes but would need a use-count table for much larger programs.

Optimization effect varies with the program: straight-line code with constants is reduced strongly (TC06: 8 → 2), while loop code whose variables change on every iteration is not reduced (TC02: 14 → 14), because the optimizer correctly forgets all values at loop labels.

6.4 Challenges Faced

- **Error recovery skipping too far.** The first version of panic-mode recovery always skipped to the next `;`. When the erroneous statement had already consumed its own `;`, the parser skipped the whole following statement and reported a confusing error on a later `}`. The fix was to synchronise only when the statement had not already ended at a `;` or `}`; the regression test `errors.mc` now checks this.
- **Correct constant folding for mixed types.** Folding `7 / 2` must give 3 for integers but a float context must not change that. Making int-to-float conversion an explicit `itof` instruction meant the optimizer always knows which arithmetic to use, so `f = 7 / 2` correctly folds to `f = 3.0`.
- **Name clashes with temporaries.** Temporaries are named `t1, t2, ...`, so a user variable called `t1` could be mistaken for a temporary by the optimizer. Such names are now rejected in semantic analysis, and the optimizer checks the full pattern `t` + digits (so a variable like

`total` is never treated as a temporary).

- **Shadowed variables in intermediate code (found by testing).** Test case TC07 showed that an inner `float x` and the outer `int x` were both written as `x` in the three-address code, so the optimizer carried the inner value 2.5 into the outer variable and produced `x = 3.5` instead of `x = 2`. The fix gives every shadowing variable a unique TAC name (`x.1`), stored in its symbol table entry; TC07 and a new regression test now confirm `x = 2`. The same review found that very long identifiers could overflow the quadruple fields, which were then enlarged.
- **Termination on any input.** Every loop in the parser must consume a token or stop; a progress check in the statement list guarantees that even input such as a stray `}` cannot cause an infinite loop.
- **Tooling on Windows.** The available GCC (6.3) needed `-std=c99`; box-drawing characters for the tree printed incorrectly in the Windows console, so the terminal tree uses ASCII; and PowerShell requires `.\` before programs in the current folder, which the build and test scripts account for.

6.5 Limitations

- **No target code.** The compiler stops at optimized three-address code; it does not generate assembly or run the program.
- **Small language.** No user-defined functions other than `main`, no parameters, arrays, pointers, strings, characters, input/output or logical operators (`&&`, `||`, `!`). A standard C program such as Hello World is therefore rejected (TC13).
- **Local optimization only.** Constant propagation does not cross labels, unreachable code (for example after a folded `goto`) is not removed, and there is no common subexpression elimination or loop optimization.
- **Simple error recovery.** Panic mode can still produce a secondary message after a lexical error (two were observed in testing, Table 18).
- **Fixed limits.** At most 2048 quadruples, 256 symbols, 100 reported errors and 63 characters per lexeme.
- **Web playground needs Python.** The browser cannot run the C compiler by itself, so the playground only works while `server.py` is running on the same computer.

6.6 Future Scope

- **Boolean expressions with short-circuit evaluation:** add `&&`, `||` and `!` and translate them

with jumps (backpatching).

- **Functions:** parameters, calls and return values, with activation records and a symbol table per function.
- **Arrays:** declarations, indexing and the address computations they require in three-address code.
- **More representations:** triples and indirect triples next to quadruples, and a DAG view of each basic block.
- **Global optimization:** data-flow analysis across basic blocks, common subexpression elimination, dead code and unreachable code elimination, and loop-invariant code motion.
- **Code generation:** translate TAC to x86 assembly or to a simple stack machine, together with an interpreter so that programs can actually be run and their results shown.
- **Fully browser-based version:** compile the C code to WebAssembly so the playground works without a server.

6.7 Conclusion

The project set out to make the phases of a compiler visible on real programs. Production compilers hide their intermediate representations, and educational tools usually cover a single phase, so students find it hard to connect lexical analysis, parsing, semantic analysis, intermediate code and optimization into one picture.

We designed a small C-like language and implemented a complete compiler front end and middle end for it in about 1336 lines of C99, with one source file per phase. The lexer recognises tokens with a longest-match rule and keyword table; the recursive descent parser builds an abstract syntax tree directly from an LL(1) grammar and recovers from syntax errors in panic mode; the semantic analyser maintains a scoped symbol table with sizes and offsets and enforces the type rules for `int` and `float`; the code generator emits three-address code as quadruples using the standard translation schemes for `if`, `while` and `for`; and the optimizer performs constant propagation, constant folding, branch folding and dead temporary removal within basic blocks.

Every phase is visualized three ways: as text in the terminal, in a self-contained HTML report with a phase status banner, tables and a collapsible tree, and in a browser playground where code can be typed and compiled with immediate feedback. The playground reuses the same C compiler through a small Python server, so what is shown is always the real output of the C code.

Testing confirmed the design. All 13 functional test cases and all 8 regression tests passed, and all 11 deliberately inserted errors were reported in the correct phase with the correct line number. The

optimizer reduced the demo program from 35 to 30 instructions without changing its meaning, and compilation remained fast, with a median of 173 ms for a 603-line program.

The main limitations are the small language and the absence of target code generation. Both are natural next steps, and the one-file-per-phase structure makes them straightforward to add.

The most important lesson for the team was how closely each phase depends on the data structure produced by the previous one: a careful AST made type checking simple, explicit type information made correct constant folding possible, and clear basic block boundaries made the optimizer safe. Building the phases by hand, rather than generating them, gave every member a working understanding of the algorithms taught in the course.

REFERENCES

[1] A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed., Pearson Addison-Wesley, 2006.

[2] K. D. Cooper and L. Torczon, *Engineering a Compiler*, 2nd ed., Morgan Kaufmann, 2011.

[3] A. W. Appel, *Modern Compiler Implementation in C*, Cambridge University Press, 1998.

[4] R. Nystrom, *Crafting Interpreters*, Genever Benning, 2021.

[5] M. E. Lesk and E. Schmidt, "Lex – A Lexical Analyzer Generator," Bell Laboratories, Computing Science Technical Report No. 39, 1975.

[6] S. C. Johnson, "Yacc: Yet Another Compiler-Compiler," Bell Laboratories, Computing Science Technical Report No. 32, 1975.

[7] C. Lattner and V. Adve, "LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation," in *Proc. International Symposium on Code Generation and Optimization (CGO)*, 2004.

[8] T. J. Parr and R. W. Quong, "ANTLR: A Predicated-LL(k) Parser Generator," *Software: Practice and Experience*, vol. 25, no. 7, pp. 789–810, 1995.

[9] M. Godbolt, "Compiler Explorer," https://godbolt.org, 2012.

[10] Free Software Foundation, "GCC Online Documentation," https://gcc.gnu.org/onlinedocs/, 2026.

[11] ISO/IEC 9899:1999, *Programming Languages – C*, International Organization for Standardization, 1999.

[12] Python Software Foundation, "http.server – HTTP servers," Python 3.12 Documentation, https://docs.python.org/3/library/http.server.html, 2026.

[13] D. E. Knuth, "On the Translation of Languages from Left to Right," *Information and Control*, vol. 8, no. 6, pp. 607–639, 1965.

[14] N. Wirth, *Compiler Construction*, Addison-Wesley, 1996.

[15] G. A. Kildall, "A Unified Approach to Global Program Optimization," in *Proc. 1st ACM Symposium on Principles of Programming Languages (POPL)*, pp. 194–206, 1973.

[16] S. H. Rodger and T. W. Finley, *JFLAP: An Interactive Formal Languages and Automata Package*, Jones and Bartlett, 2006.